Knowledge Center

What Is Application Security? Concepts, Tools & Best Practices

Application security (AppSec) helps protect application data and code against cyberattacks and data theft. It covers all security considerations during application design, development, and deployment. AppSec involves implementing software, hardware, and procedures that identify and reduce the number of security vulnerabilities and minimize the chance of successful attack.

AppSec typically involves building protections and controls into software processes. For example, automated static analysis of new code, testing new software releases for security vulnerabilities or misconfigurations, and using an application firewall to strictly define allowed and prohibited activities.

What Is Threat Modeling?

Threat modeling helps optimize the security of systems, business processes, and applications. It involves identifying vulnerabilities and objectives and defining suitable countermeasures to mitigate and prevent the impacts of threats. It is a fundamental component of a comprehensive application security program.

Here are the main steps of threat modeling:

1. Define all enterprise assets.
2. Identify the function of the application concerning the identified assets.
3. Create a security profile for each application.
4. Identify and prioritize potential threats.
5. Document adverse events and all actions taken during each scenario.

Threat models form a vital component of the security development process. When incorporated into the DevOps process, threat modeling enables teams to build security into the project during the development and maintenance phases to prevent common issues such as weak authentication, failure to validate input, lack of data encryption, and inadequate error handling.

What Is Application Security Testing?

Application security testing, or AppSec testing (AST), helps identify and minimize software vulnerabilities. This process tests, analyzes, and reports on the security level of an application as it progresses across the software development lifecycle (SDLC). It enables teams to prevent software vulnerabilities before deployment and quickly identify vulnerabilities in production. The goal is to develop stronger source code and make applications more secure.

Application Security Tools and Solutions

Here are the most common application security categories:

Static Application Security Testing (SAST)

SAST helps detect code flaws by analyzing the application source files for root causes. It enables comparing static analysis scan results with real-time solutions to quickly detect security problems, decrease the mean time to repair (MTTR), and troubleshoot collaboratively.

Dynamic Application Security Testing (DAST)

DAST is a proactive testing approach that simulates security breaches on a running web application to identify exploitable flaws. These tools evaluate applications in production to help detect runtime or environment-related errors.

Interactive Application Security Testing (IAST)

IAST utilizes SAST and DAST elements, performing analysis in real-time or at any SDLC phase from within the application. IAST tools get access to the application's code and components, which means the tools achieve the in-depth access needed to produce accurate results.

Runtime Application Security Protection (RASP)

RASP tools work within the application to provide continuous security checks and automatically respond to possible breaches. Common responses include alerting IT teams and terminating a suspicious session.

Mobile Application Security Testing (MAST)

MAST tools test the security of mobile applications using various techniques, such as performing static and dynamic analysis and investigating forensic data gathered by mobile applications. MAST tools help identify mobile-specific issues and security vulnerabilities, such as malicious WiFi networks, jailbreaking, and data leakage from mobile devices.

Web Application Firewall (WAF)

A WAF solution monitors and filters all HTTP traffic passing between the Internet and a web application. These solutions do not cover all threats. Rather, WAFs work as part of a security stack that provides a holistic defense against the relevant attack vectors.

WAF works as a protocol layer seven defense when applied as part of the open systems interconnection (OSI) model. It helps protect web applications against various attacks, including cross-site-scripting (XSS), SQL injection (SQLi), file inclusion, and cross-site forgery (CSRF).

CNAPP

A cloud native application protection platform (CNAPP) centralizes the control of all tools used to protect cloud native applications. It unifies various technologies, such as cloud security posture management (CSPM) and cloud workload protection platform (CWPP), identity entitlement management, automation and orchestration security for container orchestration platforms like Kubernetes, and API discovery and protection.

4 Application Security Best Practices

The following best practices should help ensure application security.

1. Asset Tracking

An organization must have full visibility over its assets to protect them. The first step towards establishing a secure development environment is determining which servers host the application and which software components the application contains.

Failure to track digital assets can result in hefty fines (such as Equifax's $700 million penalty for failing to protect millions of customers' data). The development and security teams must know what software runs in each app to enable timely patches and updates.

For example, Equifax could have prevented the breach by patching an Apache Struts component in a customer web portal, but they were unaware they were using the vulnerable component.

Asset tracking prevents security issues downstream. Automation can accelerate this time-consuming process and support scaling, while classification based on function allows businesses to prioritize, assess, and remediate assets.

2. Shifting Security Left

The modern, fast-paced software development industry requires frequent releases—sometimes several times a day. Security tests must be embedded in the development pipeline to ensure the Dev and security teams keep up with demand. Testing should start early in the SDLC to avoid hindering releases at the end of the pipeline.

Understanding the existing development process and relationships between developers and security testers is important to implement an effective shift-left strategy. It requires learning the teams' responsibilities, tools, and processes, including how they build applications. The next step is integrating security processes into the existing development pipeline to ensure developers easily adopt the new approach.

The CI/CD pipeline should include automated security tests at various stages. Integrating security automation tools into the pipeline allows the team to test code internally without relying on other teams so that developers can fix issues quickly and easily.

3. Performing Threat Assessments

After listing the assets requiring protection, it is possible to start identifying specific threats and countermeasures. A threat assessment involves determining the paths attackers can exploit to breach the application.

With the potential attack vectors identified, the security team can evaluate its existing security controls for detecting and preventing attacks and identify new tools to improve the company's security posture.

However, when evaluating existing security measures and planning a new security strategy, it's important to have realistic expectations about the appropriate security levels. For instance, even the highest level of protection doesn't block hackers entirely.

One consideration is the long-term sustainability of the security strategy—the highest security standards might not be possible to maintain, especially for a limited team in a growing company. Another consideration is the acceptable level of risk and a cost-benefit evaluation of the proposed security measures.

4. Managing Privileges

Not every user in an organization requires the same access privileges. Restricting access to data and applications on a need-to-know basis is a key security best practice. There are two main reasons for limiting privileges:

- If hackers can access the system with stolen credentials (e.g., from an employee in the marketing department), there must be controls to prevent them from accessing other data. Least-privilege access controls help prevent lateral movement and minimize the blast radius of an attack.
- Insider threats are more dangerous when the network has open internal access. These threats may be malicious or unintentional, such as an employee misplacing a device or downloading malicious files.

Privilege management should adhere to the principle of least privilege to prevent employees and external users from accessing data they don't need, reducing overall exposure.